Homework-3 Report

Student Name: Harun Yavaş

Student ID: 36445399294

Introduction

The objective of this project is to implement a Java-based solver for a 2D word puzzle. The solver identifies whether a specific query word exists in a character matrix by traversing neighboring cells (horizontal, vertical, and diagonal). Per the assignment requirements, the solution utilizes an explicit **Stack** data structure to perform a backtracking search (Depth-First Search - DFS) without using recursion.

Stack State Representation

To manage the search process without recursion, I implemented a helper class named **`SearchState`**. This class encapsulates the "snapshot" of the search at any given moment. Each `SearchState` object stored in the Stack contains the following information :

- **`int row, int col`**: The coordinates of the current character being visited on the board.
- **`int charIndex`**: The index of the character in the query word that matches the current cell (e.g., if searching for "JAVA" and we are at 'V', `charIndex` is 2).
- **`List<Point> path`**: A history of coordinates visited so far to form the current prefix. This ensures we can return the exact path if the word is found and prevents using the same cell twice in a single path (cycle prevention).

How the Program Works

The core logic resides in the `StudentWordPuzzleSolver` class. The algorithm proceeds in the following steps:

1. **Initialization (Raster Scan):** When `findTheWord(String word)` is called, the program first scans the entire board (row by row). Any cell matching the **first character** of the query word is converted into an initial `SearchState` and pushed onto the Stack.
2. **The Stack Loop:** The program enters a `while` loop that continues as long as the Stack is not empty.
3. **Pop and Check:** In each iteration, the top state is popped.
    - If the `charIndex` corresponds to the last character of the query word, the search is successful, and the stored `path` is returned as a `Point[]` array.
4. **Push Neighbors:** If the word is not yet complete, the algorithm looks for the next required character among the 8 neighbors. Valid neighbors are created as new `SearchState` objects and pushed onto the Stack.

Neighbor Exploration Strategy

To efficiently explore the 8 possible directions (Horizontal, Vertical, Diagonal) , I used the "Direction Array" technique rather than writing eight separate `if` statements.

Two arrays were defined:

Java
```
int[] rowOffsets = {-1, -1, -1,  0, 0,  1, 1, 1};
int[] colOffsets = {-1,  0,  1, -1, 1, -1, 0, 1};
```

By iterating through these arrays (0 to 7), the algorithm calculates the potential coordinates (`newRow`, `newCol`). A neighbor is considered valid and pushed to the Stack only if:

1. It is within the board boundaries (0 <= row < M, 0 <= col < N).
2. The character at that cell matches the *next* character in the query word.
3. The cell has not already been visited in the current path (to ensure acyclic paths).

Testing Methodology

To verify the solution, I implemented a `Main` class that performs the following tests:

1. **File Loading:** It generates a test file (`test_puzzle.txt`) formatted according to the `MatrixLoader` interface specification and verifies correct parsing.
2. **Positive Cases:** Words known to exist in the puzzle (e.g., "CAT", "JAVA") are queried, and the returned coordinates are verified against the expected grid positions.
3. **Negative Cases:** Words absent from the board are queried to ensure the method correctly returns `null`.
4. **Edge Cases:** The system was tested with words placed in reverse order and diagonal directions to ensure the 8-neighbor logic works correctly.

Assumptions and Limitations

- **Assumption:** The input file format is strictly followed (Rows first, Columns second, followed by data). However, the code includes whitespace handling to be robust against formatting errors.
- **Limitation:** Since the algorithm stores the full path history in every `SearchState` object, the memory usage could increase significantly for extremely large boards or very long words (Space Complexity). For the scope of this homework (typical NxN grids), this is negligible.